Conception et Implémentation d'un Framework de Deep Learning *From Scratch* en C99

Architecture Modulaire et Double Paradigme d'Optimisation : Rétropropagation et Algorithmes Génétiques

Hugo

2 novembre 2026

Résumé

Ce document présente la conception et le développement d'une bibliothèque de réseaux de neurones artificiels écrite entièrement en C99, sans aucune dépendance externe. Né de la volonté de s'affranchir des frameworks "boîtes noires" (tels que PyTorch ou TensorFlow) expérimentés en Python durant un parcours d'apprentissage, ce projet vise à fournir un outil modulaire, performant et réutilisable pour de futurs projets bas niveau. Le framework intègre une architecture à pointeurs opaques, un catalogue riche de fonctions d'activation et de perte, ainsi qu'une double approche d'optimisation : la rétropropagation analytique du gradient et l'optimisation par algorithme génétique.

Table des matières

1 Introduction et Contexte du Projet **2**
- 1.1 De l'expérimentation Python à l'ingénierie C . . . 2
- 1.2 Objectifs et Spécifications . . . 2

2 Fondations Théoriques et Mathématiques **2**
- 2.1 Du Neurone Formel au Perceptron Multi-Couches (MLP) . . . 2
- 2.2 Le Rôle de la Non-Linéarité . . . 2
- 2.3 Apprentissage Supervisé : Rétropropagation . . . 2
- 2.4 Alternative Évolutionniste : Algorithmes Génétiques . . . 3

3 Architecture Logicielle et Choix de Conception en C **3**
- 3.1 Encapsulation et Pointeurs Opaques . . . 3
- 3.2 Optimisation Mémoire et Localité Cache . . . 3

4 Défis Techniques et Résolution **3**

5 Validation et Résultats Expérimentaux **3**

6 Conclusion et Perspectives **4**

1 Introduction et Contexte du Projet

1.1 De l'expérimentation Python à l'ingénierie C

Plus jeune, l'expérimentation de petits projets de réseaux de neurones en Python s'est heurtée à une limite fondamentale : l'utilisation de bibliothèques haut niveau masque la réalité mathématique et architecturale du calcul sous-jacent. Pour concevoir de futurs projets personnels (simulations, systèmes autonomes, moteurs de jeux), le besoin d'une brique logicielle propre, autonome et totalement maîtrisée est apparu comme une évidence.

1.2 Objectifs et Spécifications

Le cahier des charges de ce framework s'articule autour de quatre piliers :

1. **Zéro dépendance externe (*Zero-dependency*) :** Écriture en C pur, reposant uniquement sur la bibliothèque standard (‘<stdlib.h>‘, ‘<math.h>‘, etc.).
2. **Réutilisabilité et modularité :** Offrir une API claire et découplée pour intégrer facilement des réseaux de neurones dans d'autres applications.
3. **Double paradigme d'apprentissage :** Supporter à la fois l'apprentissage supervisé par gradient (rétropropagation) et l'optimisation sans gradient par algorithme génétique.
4. **Rigueur système :** Assurer une gestion stricte de la mémoire (absence totale de fuites) et une persistance binaire robuste des modèles.

2 Fondations Théoriques et Mathématiques

2.1 Du Neurone Formel au Perceptron Multi-Couches (MLP)

Le modèle repose sur le neurone formel, effectuant une combinaison linéaire suivie d'une non-linéarité :

$$z_j^{(l)} = \sum_i W_{ji}^{(l)} a_i^{(l-1)} + b_j^{(l)} \quad \text{et} \quad a_j^{(l)} = \sigma(z_j^{(l)}) \tag{1}$$

Pour s'affranchir de la limite historique du XOR (incapacité du perceptron simple à séparer des données non-linéaires), les neurones sont organisés en couches successives (*Multi-Layer Perceptron*). La propagation avant s'écrit sous forme matricielle :

$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}) \tag{2}$$

2.2 Le Rôle de la Non-Linéarité

Sans fonction d'activation non-linéaire, l'enchaînement de plusieurs couches se réduit mathématiquement à une unique transformation linéaire équivalente. L'intégration de fonctions telles que la Sigmoïde, Tanh ou ReLU confère au réseau sa capacité à modéliser des frontières de décision complexes.

2.3 Apprentissage Supervisé : Rétropropagation

L'apprentissage par gradient repose sur la minimisation d'une fonction de coût (ex : Erreur Quadratique Moyenne - MSE). Par application de la règle de dérivation en chaîne (*Chain Rule*), les sensibilités (deltas) sont propagées de la sortie vers l'entrée pour mettre à jour les poids via la descente de gradient stochastique (SGD) :

$$W^{(l)} \leftarrow W^{(l)} - \alpha \frac{\partial L}{\partial W^{(l)}} \tag{3}$$

2.4 Alternative Évolutionniste : Algorithmes Génétiques

Lorsque le gradient n'est pas disponible ou adapté, l'approche évolutionniste prend le relais. Une population de réseaux est évaluée via une fonction de fitness générique. Les meilleurs individus sont sélectionnés par tournoi, protégés par élitisme, puis combinés par croisement (*crossover*) et soumis à des mutations stochastiques.

3 Architecture Logicielle et Choix de Conception en C

3.1 Encapsulation et Pointeurs Opaques

Pour garantir l'intégrité des structures internes et offrir une API publique propre, la bibliothèque utilise des pointeurs opaques définis dans 'nn.h' et implémentés en interne via des structures détaillées ('internal.h').

```c
typedef struct {
    int input_dim;
    int output_dim;
    double *weights;  // Tableau plat 1D
    double *biases;
    double *output;
    double *grad_weights;
    double *grad_biases;
} Layer;

struct NeuralNetwork {
    int num_layers;
    Layer *layers;
    LossType loss;
};
```

Listing 1 – Structure interne d'une couche et d'un réseau

3.2 Optimisation Mémoire et Localité Cache

Plutôt que d'utiliser des tableaux de tableaux 2D (générateurs de fragmentation mémoire et de sauts de cache inefficaces), les poids de chaque couche sont stockés dans un tableau plat 1D de taille 'input_dim * output_dim'. Cette disposition garantit une excellente localité cache lors des multiplications matricielles et simplifie grandement la sérialisation binaire directe via 'fwrite' et 'fread'.

4 Défis Techniques et Résolution

— **Gestion du cycle de vie :** Mise en place de patrons stricts d'allocation et de libération en cascade ('NN_Create', 'NN_Free', 'NN_Population_Free') pour éliminer tout risque de fuite mémoire.
— **Sécurisation de l'algorithme génétique :** Évitement des pointeurs pendants (*dangling pointers*) lors de la mise à jour générationnelle des populations, et correction du tri par index pour dissocier le classement des scores de la manipulation des pointeurs de réseaux.

5 Validation et Résultats Expérimentaux

Le framework a été validé sur le cas d'école du problème du XOR :

— **Rétropropagation :** Convergence atteinte en [X] époques avec un taux d'apprentissage de [X].
— **Algorithme Génétique :** Résolution validée avec une population de [100-200] individus, un taux de mutation de [X] et une convergence en [X] générations.
— **Précision finale :** 100% de réussite sur l'ensemble des cas logiques.

6 Conclusion et Perspectives

Ce projet démontre qu'il est possible de concevoir un framework de Deep Learning performant et modulaire en C pur. En combinant rigueur mathématique et maîtrise des systèmes bas niveau, la bibliothèque est désormais prête à être intégrée dans de futurs projets personnels. Les pistes d'évolution incluent la parallélisation de l'évaluation des populations via OpenMP.